SAP CAP Service CI/CD Guide for Azure DevOps

Automated Build, Package, and Deployment of SAP Cloud Application Programming Model (CDS) to SAP BTP Cloud Foundry using MTA Architectures

1. Overview & Architecture Strategy

This technical guide establishes an enterprise-grade Continuous Integration and Continuous Deployment (CI/CD) pipeline in Azure DevOps specifically designed for **SAP Cloud Application Programming Model (CAP)** node services utilizing `mta.yaml`. The pipeline automates CDS compilation, dependency management, MTA building, and multi-space SAP BTP Cloud Foundry deployment.

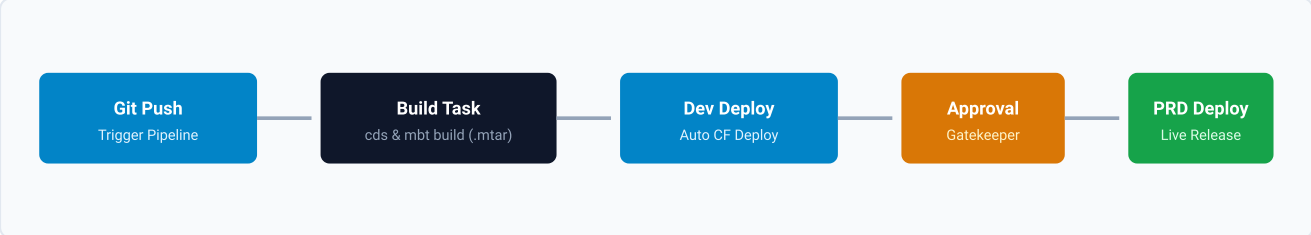
> **Critical SAP BAS Dependency Fix:** Projects developed in SAP Business Application Studio (BAS) often lock cache URLs pointing to internal SAP proxy mirrors (`http://nginx-redirector...`) inside `package-lock.json`. Ensure all lockfile registry URLs are replaced with standard `https://registry.npmjs.com` before triggering Azure pipelines.

2. Prerequisites & Environment Blueprint

Ensure the target SAP BTP Subaccount and Azure DevOps project are configured according to the following architecture map:

Component	Required Resource	Configuration Purpose
SAP BTP Cockpit	Subaccount & Cloud Foundry Spaces	Provision distinct deployment spaces: `dev` (Development) and `prd` (Production).
CAP Repository	Root Directory Content	Must contain valid `mta.yaml`, `package.json`, `db/`, and `srv/` CDS definitions.
Agent Runner	Hosted or Self-Hosted Agent	Microsoft Ubuntu Agent (`ubuntu-latest`) or On-Premises Private Agent for SAP SolMan/ABAP.
BTP Auth	Deployment User / Service Key	BTP credentials with `Space Developer` role assigned to target subaccount spaces.

3. High-Level Flow Chart



4. Continuous Integration (CI) — YAML Pipeline Definition

Create an `azure-pipelines.yml` file in your CAP project root. This configuration sets up Node.js, installs the SAP CDS Compiler (`@sap/cds-dk`) and Cloud MTA Build Tool (`@sap/mbt`), compiles your CDS models, and outputs the deployable `.mtar` archive.

```
      # Azure DevOps YAML Pipeline for SAP CAP CDS Application trigger: branches: include: - main - master pool: vmImage: 'ubuntu-latest' variables: buildOutputDir: '$(Build.ArtifactStagingDirectory)' steps: - task: NodeTool@0 inputs: versionSpec: '18.x' displayName: 'Use Node.js 18.x' - script: | npm install -g @sap/cds-dk @sap/mbt displayName: 'Install Global Dependencies (@sap/cds-dk & @sap/mbt)' - script: | npm ci displayName: 'Install Project NPM Packages' - script: | mbt build -p=cf -s=. -t=$(buildOutputDir) displayName: 'Build SAP CAP MTA Archive (.mtar)' - task: PublishPipelineArtifact@1 inputs: targetPath: '$(buildOutputDir)' artifact: 'cap-mta-artifact' publishLocation: 'pipeline' displayName: 'Publish MTA Artifact to Pipeline Workspace'
```

5. Alternative Classic Pipeline Editor Configuration

If your team utilizes the Classic UI Editor instead of YAML definitions, configure the following sequence:

- **Navigation:** Pipelines → New Pipeline → Click *"Use the classic editor"* → Select Source Repository & Branch.
- **Task 1 (Command Line):** `npm install -g @sap/cds-dk @sap/mbt`
- **Task 2 (Command Line):** `mbt build -p=cf -s=$(Build.SourcesDirectory) -t=$(Build.ArtifactStagingDirectory)`
- **Task 3 (Publish Pipeline Artifact):** Set Path to `$(Build.ArtifactStagingDirectory)` and Artifact Name to `cap-mta-artifact`.

> **Continuous Integration Trigger:** In Classic Mode, navigate to the **Triggers** tab and check *"Enable continuous integration"* to ensure commits to `master` automatically start the build process.

6. Continuous Deployment (CD) — Release Pipeline Setup

Configure an Azure DevOps **Release Pipeline** to listen for newly published build artifacts and execute SAP Cloud Foundry deployments automatically across spaces.

Step A: Environment Variables Setup

Define the following secure pipeline parameters in the **Variables** tab of your Release Pipeline:

Variable Name	Sample Value	Scope	Description
`api-url`	`https://api.cf.eu10.hana.ondemand.com`	Release	SAP BTP Cloud Foundry Regional Endpoint
`cf-user`	`deployer-service-account@company.com`	Release	BTP Deployment User Username
`dev-org` / `prd-org`	`my-btp-org-name`	Stage Specific	Cloud Foundry Organization Name
`dev-space` / `prd-space`	`dev` / `prd`	Stage Specific	Target BTP Cloud Foundry Space
`dev-password` / `prd-password`	•••••••••••• (Secret 🔒)	Stage Specific	User Password or Service Account Token

Step B: Stage Task Sequence (`Dev` & `Prd`)

Add the following command scripts inside each Release Stage (`Dev` and `Prd`):

```
# Task 1: Download Build Artifact # Task Type: Download Pipeline Artifacts -> Select 'cap-mta-artifact' # Task 2: Install MultiApps Plugin (Command Line) cf install-plugin multiapps -f # Task 3: Login to SAP Cloud Foundry Target Space (Command Line) cf login -a $(api-url) -u $(cf-user) -p $(dev-password) -o $(dev-org) -s $(dev-space) # Task 4: Execute MTA Deployment (Command Line) cd $(Pipeline.Workspace)/cap-mta-artifact cf deploy $(ls *.mtar | head -n 1) -f
```

7. Quality Gates & Governance Controls

> **Pre-Deployment Approvals:** Click the pre-deployment trigger (person icon) on the `Prd` stage. Enable **Pre-deployment approvals** and specify lead architects or release managers. Production deployment will pause safely until explicit sign-off is logged in Azure DevOps.